

Spring Cache

Spring Cache

메소드 호출 결과를 캐시에 저장해두고, 다음에 동일한 파라미터로 호출될 때 캐시에 저장된 값을 반환하는 메커니즘이다.

Spring Cache는 AOP 기반의 메소드 레벨 캐싱을 제공한다. 개발자가 캐시 저장, 조회, 삭제 로직을 직접 구현할 필요 없이 어노테이션만으로 캐싱 기능을 적용할 수 있다.

1. 사용 목적과 특징

장점

- **DB 및 외부 API 호출 횟수 감소**: 반복적인 데이터 조회 시 캐시를 활용하여 데이터베이스나 외부 시스템에 대한 요청을 줄일 수 있다.
- **반복 조회 API 성능 향상**: 동일한 요청에 대해 캐시된 결과를 즉시 반환하여 응답 속도가 크게 개선된다.
- **코드 간결화**: 캐시 로직을 비즈니스 로직과 분리하여 코드의 가독성과 유지보수성이 향상된다.

단점

- **캐시 일관성 관리 필요**: 데이터가 변경되었을 때 캐시를 적절히 갱신하지 않으면 **stale** 데이터를 반환할 수 있다.
- **캐시 정책 이해 필요**: 캐시의 생성, 갱신, 만료 시점을 정확히 이해하고 설정해야 한다.
- **잘못된 사용 시 부작용**: 캐시 설정이 부적절하면 오래된 데이터를 반환하거나 메모리 누수가 발생할 수 있다.

2. 주요 어노테이션 사용법

2.1 @Cacheable: 조회 및 저장

```
@Cacheable(cacheNames = "users", key = "#id")
public User getUser(Long id) {
    // 구현 로직
}
```

- 캐시에 지정된 Key(**#id**)가 존재하면 데이터베이스에 접근하지 않고 캐시된 값을 반환한다.
- 캐시에 데이터가 없으면 메소드를 실행하여 결과를 캐시에 저장한 후 반환한다.

2.2 @CachePut: 강제 갱신

```
@CachePut(cacheNames = "users", key = "#user.id")
public User updateUser(User user) {
    // 구현 로직
}
```

- 메소드를 매 호출마다 실행하며, 그 결과를 무조건 캐시에 저장(갱신)한다.
- 주로 데이터 수정 API에서 사용하여 캐시를 최신 상태로 유지한다.

2.3 @CacheEvict: 캐시 삭제

```
@CacheEvict(cacheNames = "users", key = "#id")
public void deleteUser(Long id) {
    // 구현 로직
}
```

- 지정된 캐시 엔트리를 삭제한다.
- 주로 데이터 삭제 API에서 사용하여 해당 캐시를 제거한다.
- `allEntries = true` 옵션을 사용하면 해당 캐시의 모든 엔트리를 삭제할 수 있다.

2.4 @Caching: 복합 처리

```
@Caching(
    put = @CachePut(cacheNames = "users", key = "#user.id"),
    evict = @CacheEvict(cacheNames = "user_list", allEntries = true)
)
public User updateUser(User user) {
    // 구현 로직
}
```

- 여러 캐시 어노테이션을 하나의 메소드에 동시에 적용할 때 사용한다.
- 위 예시는 특정 사용자 캐시를 갱신하면서 동시에 사용자 목록 캐시를 모두 삭제하는 경우이다.

3. Spring Cache 내부 동작 원리

Spring Cache는 **프록시 기반 AOP**를 활용하여 동작한다. 스프링은 `@Cacheable` 등의 어노테이션이 적용된 메소드 앞뒤로 캐시 처리 로직을 자동으로 삽입한다.

동작 흐름

1. 클라이언트가 프록시 객체의 메소드를 호출
2. 프록시가 캐시 저장소에서 해당 키를 조회

3. 캐시 히트(Hit): 캐시된 값을 즉시 반환하고 원본 메소드는 실행하지 않음
4. 캐시 미스(Miss): 원본 메소드를 실행
5. 메소드 실행 결과를 캐시에 저장
6. 클라이언트에 결과 반환

프록시 우회 주의사항

같은 클래스 내부에서 메소드를 직접 호출하는 경우, 프록시를 거치지 않기 때문에 AOP가 적용되지 않는다. 즉, 캐시 어노테이션이 동작하지 않으므로 이러한 패턴은 피해야 한다.

```
// 잘못된 예시
public class UserService {

    @Cacheable("users")
    public User getUser(Long id) {
        // ...
    }

    public User findUser(Long id) {
        return this.getUser(id); // 프록시 우회로 캐시 동작 안 함
    }
}
```
